PSTAT 10 Data Science Principles

Lecture 2: Vectors

John Robin Inston
johninston@ucsb.edu
University of California, Santa Barbara

August 4, 2026

Introduction

🔁 Review: Lecture 1

👈 Last lecture we started by going through course logistics 🤭 before looking into:

- The programming language **R**
- The IDE **RStudio**
- **Quarto** documents

We then looked at some basic programming concepts such as:

- Data types
- R calculations
- The assignment operator (`<-`)

👀 Overview: Lecture 2

👇 This lecture we continue our introduction, looking at

- Data structures
- Filtering
- Recycling
- Sorting
- Vectorization

Data Structures

🔷 Scalars

What is a scalar?

- A **scalar** data form is an object holding **one value**.
  - For example, from last lecture:

```
1 x <- 16 # numeric scalar
2 y <- "PSTAT 10" # character string scalar
3 z <- FALSE # Boolean scalar
```

Vectors, Matrices and Lists

- Often we need to use different data structures that contain **multiple values** of **multiple dimensions**
  - This including (possibly) a **variety of data types**.

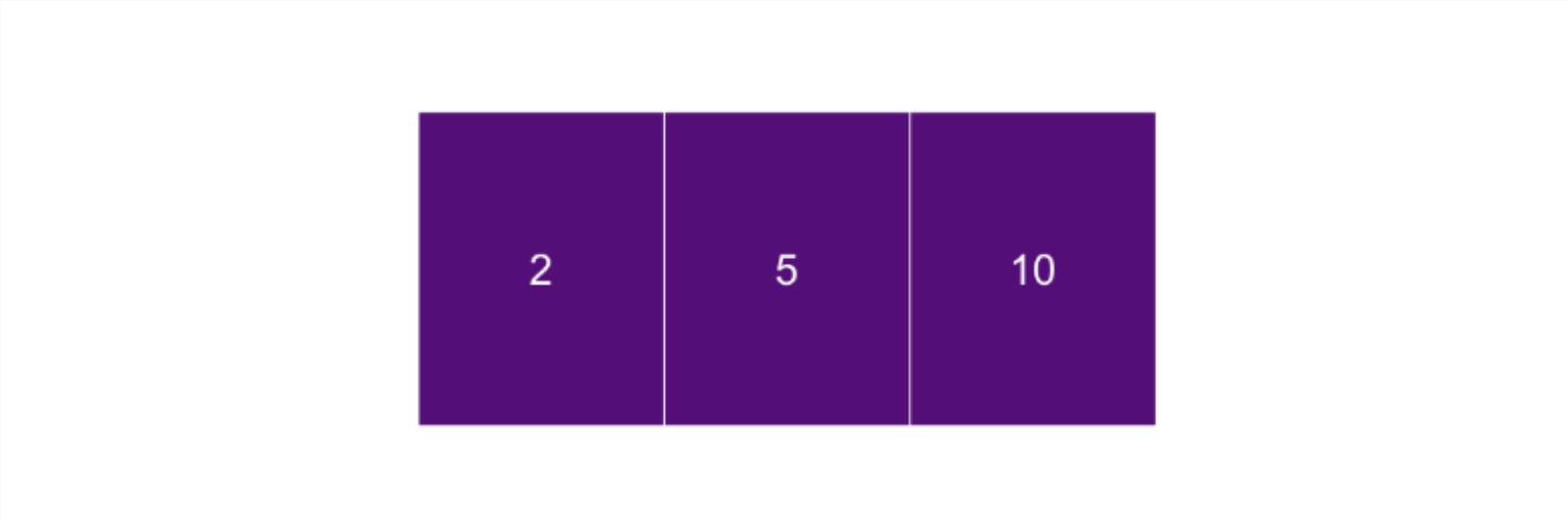
- **Vectors**
- **Matrices**
- **Data frames**
- **Lists**

$$\vec{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}; \qquad \mathbf{X} = \begin{bmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{np} \end{bmatrix}; \qquad L = \{x, \vec{x}, X\}.$$

Vectors

What is a vector?

- A **vector** data form is an object holding **multiple values all of the same datatype**. — For example all values are numeric, or all strings.
    - We therefore say that vectors are **atomic**.



Vector Diagram

> There are a number of different ways of constructing vector objects in R.

Creating Vectors I

Combine Function `c()`

- The simplest way to create a vector in R is using the **combine function** `c()`.
  - This function takes a **comma separated list** of values and combines them into a vector.
  - Use the `help` function `?c` to learn more about the `c()` function.

```r
# numeric vector
num_vec <- c(1,2,3,4,5)
print(num_vec)
```

```
[1] 1 2 3 4 5
```

```r
# character string vector
char_vec <- c("Dog", "Cat", "Parrot", "Monkey")
print(char_vec)
```

```
[1] "Dog"    "Cat"    "Parrot" "Monkey"
```

```r
# Boolean vector
bool_vec <- c(TRUE, TRUE, FALSE, FALSE, TRUE)
print(bool_vec)
```

```
[1]  TRUE  TRUE FALSE FALSE  TRUE
```

- ! Note that all elements of a vector must be of the **same data type**.
  - If you try to combine different data types, R will **coerce** the values into a single data type (discussed later in this lecture).

Creating Vectors II

🔁 `rep()` Function

- The **repeat function** `rep()` **repeats a value** a specified number of times.

```r
# repeating 3 over 7 values
rep_1 <- rep(3, 7)
rep_1
```

```
[1] 3 3 3 3 3 3 3
```

```r
# repeating "dog" over 4 values
rep_2 <- rep("dog", 4)
rep_2
```

```
[1] "dog" "dog" "dog" "dog"
```

🔢 `seq()` Function

- The **sequence function** `seq()` creates a vector of **consecutive values**.
  - There are a number of different combinations of arguments that can be used to create a vector using `seq()`.

```r
seq_1 <- seq(from = 1, to = 10, by = 1); seq_1
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```r
seq_2 <- seq(from = 1, to = 10, length.out = 5); seq_2
```

```
[1]  1.00  3.25  5.50  7.75 10.00
```

🧩 Combining Methods

Flexibity of Vector Creation

- You can **combine methods** to efficiently produce any vector you need.
  - Simply **nest** the functions inside one another.
- For example, to create a vector of 12 values consisting of the numbers 1 through 4 repeated 3 times:

```
1  comb_1 <- rep(c(1,2,3,4), 3)
2  comb_1
```

```
[1] 1 2 3 4 1 2 3 4 1 2 3 4
```

- Or to create a vector of 2 TRUE values followed by 3 FALSE values:

```
1  comb_2 <- c(rep(TRUE, 2), rep(FALSE, 3))
2  comb_2
```

```
[1]  TRUE  TRUE FALSE FALSE FALSE
```

- Or to create a vector of numbers from 1 to 3 in increments of 0.25, followed by two 1.7 values:

```
1  comb_3 <- c(seq(1,3,by=0.25), rep(1.7,2))
2  comb_3
```

```
[1] 1.00 1.25 1.50 1.75 2.00 2.25 2.50 2.75 3.00 1.70 1.70
```

> Try to always think about the most **efficient** way to create a vector, as there are often multiple ways to do so!

Vector Property Functions

📏 `length()` Function

- If we want to know how many values are in a vector we can use the `length()` function.

```
1 length(comb_3)
```

```
[1] 11
```

```
1 length(rep_2)
```

```
[1] 4
```

🏷️ `typeof()` Function

- To find the **datatype** of our vector we use the `typeof()` function:

```
1 typeof(bool_vec)
```

```
[1] "logical"
```

```
1 typeof(comb_1)
```

```
[1] "double"
```

`is.type` Functions

- We can also use the `is.numeric()`, `is.character`, and `is.logical` functions to check the datatype of a vector:

```
1 is.numeric(comb_1)
```

```
[1] TRUE
```

Double Datatype

> Did anybody notice something interesting about the `typeof()` function output for `comb_1`?

```
1 print(comb_1)
```

```
[1] 1 2 3 4 1 2 3 4 1 2 3 4
```

```
1 typeof(comb_1)
```

```
[1] "double"
```

What is a double?

- Numeric datatypes are split into two sub-types:
  - **Integer type** — all integer values.
  - **Double type** - a numeric datatype that can store decimal values.
    - Equivalently, a double is a **floating point number**.

```
1 typeof(seq(1,6))
```

```
[1] "integer"
```

```
1 typeof(seq(1,8, by = 2))
```

```
[1] "double"
```

> Typically, numeric data is **by default saved as a double** unless we specify that it should be saved as an integer using the `as.integer()` function.

💪 Exercise — Coding Vectors

For the next 2 minutes consider the following example.

Your somewhat unhelpful team mate on a class project reports the result of their survey of 20 members of the public about their pets:

> The first 10 people I asked said they had dogs, then it was back and forth between cat and dog for 8 people, and the last two were fish.

Create a vector called `pets` that contains the 20 values of the survey results.

02:00

✅ Solution — Coding Vectors

Combining Methods

- The easiest way to create the vector is to combine the `rep()` and `c()` functions:

```
pets <- c(rep("dog", 10), rep(c("cat", "dog"), 4), rep("fish", 2))
print(pets)
```

```
 [1] "dog"  "dog"  "dog"  "dog"  "dog"  "dog"  "dog"  "dog"  "dog"  "dog"
[11] "cat"  "dog"  "cat"  "dog"  "cat"  "dog"  "cat"  "dog"  "fish" "fish"
```

Side Note on Syntax

- Here we have nested functions inside each other and our code is both a bit long and hard to read.
  - To make it easier to read we can use line breaks and indentation to make the code more clear:

```
pets <- c(
  rep("dog", 10),
  rep(c("cat", "dog"), 4),
  rep("fish", 2)
)
```

- Some standard conventions:
  - Start new function arguments on a new line and to indent them (use `Tab` key in RStudio).
  - If your code is longer than 80 characters, consider breaking it up into multiple lines.

Vectorization

⚡ Vectorization

What is vectorization?

- One of the most powerful features of R is its ability to perform **vectorized computations**.
  - This means that we can perform operations on entire vectors at once.
  - Vectorized computations are **fast**, as they are much more **efficient** than alternatives such as looping (discussed in Lecture 4).

Vectorized Arithmetic

- Recall that we can perform mathematical operations on scalars:

```r
# mathematical operations on scalars
a <- 5
a^3 - 2
```

```
[1] 123
```

- We can perform mathematical operations on vectors in the same way, and R will **apply the operation to each element** of the vector:

```r
# Recall sequence 1
print(seq_1)
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```r
# mathematical operations
seq_1^3 - 2
```

```
 [1]  -1   6  25  62 123 214 341 510 727 998
```

💪 Exercise - Vectorized Computations

Suppose a class of 5 students have **midterm scores** of `73, 82, 86, 86, 94` and **final exam scores** `85, 71, 83, 92, 92`.

1. Create two vectors named `midterm_scores` and `final_scores` storing the midterm and final score values.
2. Create a vector named `final_grade` consisting of the students' final grades which are calculated as follows:

$$\text{final} = \frac{(2 \times \text{midterm}) + (3 \times \text{final})}{5}.$$

03:00

✅ Solution — Coding Vectors

Defining the Vectors

- Use combine function `c()` to define the vectors:

```r
# define the vectors
midterm_scores <- c(73, 82, 86, 86, 94)
final_scores <- c(85, 71, 83, 92, 92)
```

Calculating Final Grades

- Use vectorized arithmetic to calculate the final grades:

```r
# calculate final grades
final_grade <- (2*midterm_scores + 3*final_scores)/5
print(final_grade)
```

```
[1] 80.2 75.4 84.2 89.6 92.8
```

Indexing

🎯 Accessing Vector Data

What is indexing?

- If we need to **access specific elements** of a vector we do so by **indexing**.
  - In `R` we use braces `vec_1[...]`.

```r
# recall the vectors we created earlier
print(comb_1)
print(comb_2)
print(comb_3)
```

```
[1] 1 2 3 4 1 2 3 4 1 2 3 4
```

```
[1]  TRUE  TRUE FALSE FALSE FALSE
```

```
 [1] 1.00 1.25 1.50 1.75 2.00 2.25 2.50 2.75
3.00 1.70 1.70
```

- We can index specific values as follows:

```r
# index the 4th value of comb_3
comb_3[4]
```

```
[1] 1.75
```

- We can also index multiple values by providing a vector of indices:

```r
# index multiple values
comb_2[c(1,3,5)]
```

```
[1]  TRUE FALSE FALSE
```

```r
comb_1[2:8] # NOTE: use of colon operator (look up)
```

```
[1] 2 3 4 1 2 3 4
```

🎯 Negating & Logical Indexing

Specifying what we don't want! 👎

- We can index **everything but certain values** by simply **negating our index** value / vector:

```
# Index everything but the 2nd value of comb_2
comb_2[-2]
```

```
[1]  TRUE FALSE FALSE FALSE
```

```
# Index everything but 2nd through 8th values of comb_3
comb_3[-(2:8)] # careful to use parenthesis
```

```
[1] 1.0 3.0 1.7 1.7
```

Boolean Indexing

- We can also **index using logical statements**:

```
comb_2[c(FALSE, FALSE, TRUE, TRUE, TRUE)]
```

```
[1] FALSE FALSE FALSE
```

- This is specifying that we want values 3, 4, and 5 of `comb_2` because the logical vector is `TRUE` for those indices.

> ❗ This will become very important when we look at filtering later in this lecture.

Named Vector Indices

From numbers to names

- For convenience we can **assign names to each vector index**.
- The `names()` function can be used to both **inspect the current names of vector indices**, and **assign new names to vector indices**.

Grading Excercise Example

- Suppose the students from our exercise on grades a few slides ago (in the correct order) are named: `Alex`, `Becca`, `Ciar`, `Dobby`, and `Eliah`.
- We can inspect the current index names using the `names()` function (currently empty).

```
1 names(final_scores)
```

```
NULL
```

- Assign new names to the vector indices using the `names()` function with the assignment operator:

```
1 names(final_scores) <- c("Alex", "Becca", "Ciar", "Dobby", "Eliah")
2 names(final_scores)
```

```
[1] "Alex"  "Becca" "Ciar"  "Dobby" "Eliah"
```

```
1 # Indexing using names
2 final_scores[c("Alex", "Dobby")]
```

```
Alex Dobby
  85    92
```

Filtering

🔍 Filtering

What is filtering?

- Instead of indexing specific values we can **return all values that meet some logical criteria**
  - This is called **filtering**.
  - We input some **logical expression** into the index and **any value for which the expression is true is returned**.
- For example, to return the final grades of students who scored **85% or higher** on their final grade:

```
1  final_grade[final_grade >= 85]
```

```
[1] 89.6 92.8
```

🤔 Recall Boolean Indexing

- Notice that the logical expression is creating the following logical vector:

```
1  final_grade >= 85
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE
```

- Our indexing is returning the **last two elements**.

💪 Exercise — Handling Vectors

For the next 3 minutes complete the following exercise.

The following 8 values are an individual's monthly grocery bill from January through August:

`326, 281, 287, 308, 405, 299, 421, 301`

1. Create a vector called `grocery` with the 8 values. Check that all 8 are accounted for.
2. Name the **vector indices** as `Jan, Feb, Mar, Apr, May, Jun, Jul, Aug`.
3. Determine how many months had a grocery bill **exceeding 300**.
4. Determine how much was spent on groceries in `Jan, Mar` and `Aug` **combined**.

03:00

✅ Solution — Handling Vectors

Define the Vector

```r
# define grocery vector
grocery <- c(326, 281, 287, 308, 405, 299, 421, 301)
```

Label the Indices

```r
# label indices
names(grocery) <- c("Jan", "Feb", "Mar", "Apr", "May", "June", "Jul", "Aug")
```

Compute the Number of Months Exceeding 300

```r
# determine how many months had a grocery bill > 300
length(grocery[grocery > 300])
```

```
[1] 5
```

Compute the Sum of Selected Months

```r
# determine how much was spent on groceries in months
sum(grocery[c("Jan", "Mar", "Aug")])
```

```
[1] 914
```

Recycling

♻️ Recycling

What is Recycling?

- We have seen the result of **adding two numeric vectors of the same length**:

```
1  c(1,2) + c(3,4)
```

```
[1] 4 6
```

- When we add two numeric vectors of **different length**, R will **recycle** values on the shorter vector (**looping from start to finish**) until the computation is complete:

```
1  c(1,2) + rep(1,7)
```

```
[1] 2 3 2 3 2 3 2
```

🤔 Recall Vectorized Arithmetic

- Recycling is actually what let us subtract a **single scalar** from an entire vector earlier in the lecture:

```
1  seq_1 - 2
```

```
[1] -1  0  1  2  3  4  5  6  7  8
```

- R silently **recycles the length-1 vector** `2` to match the length of `seq_1`, subtracting it from every element.

! When Recycling Goes Wrong

Multiples of Lengths

- Recycling only works **cleanly** when the longer length is a **whole-number multiple** of the shorter length.
    - If it isn't, R still recycles — but warns you:

```
1  c(1,2,3) + c(1,2)
```

```
Warning in c(1, 2, 3) + c(1, 2): longer object length is not a multiple of
shorter object length
```

```
[1] 2 4 4
```

> 📌 The computation still runs, but this is a common source of silent bugs — always **double-check your vector lengths** before combining them.

- Hint: If in doubt, the **modulo operator** `%%` tells you if one length divides evenly into the other — a **non-zero** result means expect a warning.

```
1  length(c(1,2,3)) %% length(c(1,2))
```

```
[1] 1
```

- As a general rule it is easier to try to avoid recycling!

Coercion

🔀 What is Coercion?

What is Coercion?

- **Vector coercion** happens when R changes a vector's data type because you're **combining different data types**.
- For example, if we try to create a vector consisting of both character strings and numeric values:

```
1 combined_vec <- c("cat", "dog", 2, "monkey", 7, 9)
2 combined_vec
```

```
[1] "cat"    "dog"    "2"      "monkey" "7"      "9"
```

- Notice the new vector consists of **only character strings** — even the numbers `2`, `7` and `9` were converted:

```
1 typeof(combined_vec)
```

```
[1] "character"
```

🔀 Examples of Coercion

More Combinations

- The same thing happens whenever we mix **logical**, **numeric** or **character** values in one vector:

Logical & Numeric Values

```
1 bool_num_vec <- c(TRUE, FALSE, TRUE, 12, 5, 6)
2 bool_num_vec
```

```
[1]  1  0  1 12  5  6
```

- Now let's try combining all three types together:

Logical, Numeric & Character Strings

```
1 triple_vec <- c("dog", TRUE, FALSE, 7, 6, "cat")
2 triple_vec
```

```
[1] "dog"   "TRUE"  "FALSE" "7"     "6"     "cat"
```

> 📌 We notice an order of priority: (1) **Character strings**; (2) **Numeric**; and (3) **Boolean**.

❗ Why Coercion Matters

A Common Gotcha

- Coercion can silently break your calculations if you're not paying attention:

```
1 nums <- c("1", "2", 3)
2 nums + 1
```

```
Error in nums + 1: non-numeric argument to binary operator
```

> 📌 Because `nums` was coerced to **character**, R can no longer do arithmetic on it — always check `typeof()` if a calculation errors unexpectedly.

- Hint: use `as.numeric()`, `as.character()`, or `as.logical()` to **explicitly convert** a vector back to the type you need.

Sorting

🔃 Sorting

The `sort()` Function

- When dealing with vectors we often wish to **sort** them into some kind of order — we can do so using the `sort()` function.

```
1  sort(c(5,23,7,87,45,65)) # numerical sorting
```

```
[1]  5  7 23 45 65 87
```

- The same function works on character vectors — sorted **alphabetically**, with an optional `decreasing = TRUE` argument:

```
1  sort(c("a", "g", "b", "e", "z"), decreasing = TRUE) # alphabetical sorting
```

```
[1] "z" "g" "e" "b" "a"
```

- And for logical vectors, `FALSE` is always treated as **smaller** than `TRUE`:

```
1  sort(c(TRUE, FALSE, TRUE, FALSE, TRUE, FALSE)) # logical sorting
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE  TRUE
```

🔢 The `order()` Function

Sorting One Vector by Another

- `sort()` only reorders the **values of a single vector** — but what if we want to reorder a **second, related vector** to match?
- The `order()` function returns the **indices** that would sort a vector, rather than the sorted values themselves:

```
1 scores <- c(73, 95, 88, 60)
2 order(scores)
```

```
[1] 4 1 3 2
```

- We can then use these indices to **reorder a different vector** — for example, a vector of the corresponding student names:

```
1 students <- c("Alex", "Becca", "Ciar", "Dobby")
2 students[order(scores)]
```

```
[1] "Dobby" "Alex"  "Ciar"  "Becca"
```

> 📌 This is the standard way to keep **multiple related vectors in sync** when sorting by one of them.

💪 Exercise — Final Problem

Spend the last 5 minutes completing the following exercises, combining all of the skills you have developed throughout the lecture.

1. Define the vector `num_seq` as a vector of consecutive integers starting at 1 and ending at 500.
2. Calculate the sum of the square roots of all numbers between 200 and 400.
3. Calculate the sum of the natural logarithm of every even number between 1 and 500.
4. Create a vector named `rev_seq` which consists of all numbers between 1 and 5 and between 495 and 500, displayed in descending order.

05:00

✅ Solution — Final Problem

Define the sequence

```r
# define the sequence
num_seq <- seq(1, 500, by = 1)
```

Sum the square roots

```r
# compute the sum of sqrts
sum(sqrt(num_seq[200:400]))
```

```
[1] 3464.785
```

Sum of natural logs

```r
# compute the sum of natural logs
sum(log(num_seq[c(FALSE, TRUE)]))
```

```
[1] 1307.332
```

Create the reverse sequence

```r
# create vector
rev_seq <- sort(c(num_seq[1:5], num_seq[495:500]), decreasing = TRUE); rev_seq
```

```
 [1] 500 499 498 497 496 495   5   4   3   2   1
```

Summary

✅ Topics Covered

- Data structures
- Indexing
- Filtering
- Recycling
- Coercion
- Sorting
- Vectorization

📅 Next Class

- Matrices
- Arrays
- Lists